TRAIN SIM CLASSIC · VEHICLE WORKFLOW GUIDE

Part 1

What a TSC vehicle actually is

The game never sees your Blender model. It sees a bundle of blueprints that point at geometry, animations, curves and scripts, tied together by named nodes and named control values. Understand that bundle and every rule in the later parts makes sense.

In this part

1. A vehicle is a graph of files, not a model
2. The components inside a vehicle blueprint
3. How the game places a vehicle on the track (side view with real numbers)
4. Three coordinate systems and how they map
5. Control values: the bus everything talks through
6. The traction and braking chain
7. Engine vs wagon vs tender
8. What happens when a scenario loads
9. The glue: names

Examples use the Kuju “Sample Electric Engine” (`RailVehicles\Electric\EngineSample`) and “Example Steam Engine” (SDJR 7F 2-8-0) blueprints supplied with this project.

1 · A vehicle is a graph of files, not a model

When you place a locomotive in a scenario, TSC loads one **vehicle blueprint** (an engine, wagon or tender blueprint). That file contains almost no geometry. Instead it **references** other files by path, and those files reference more files. The figure below is the real graph of the Kuju sample electric locomotive, with its actual file names. verified

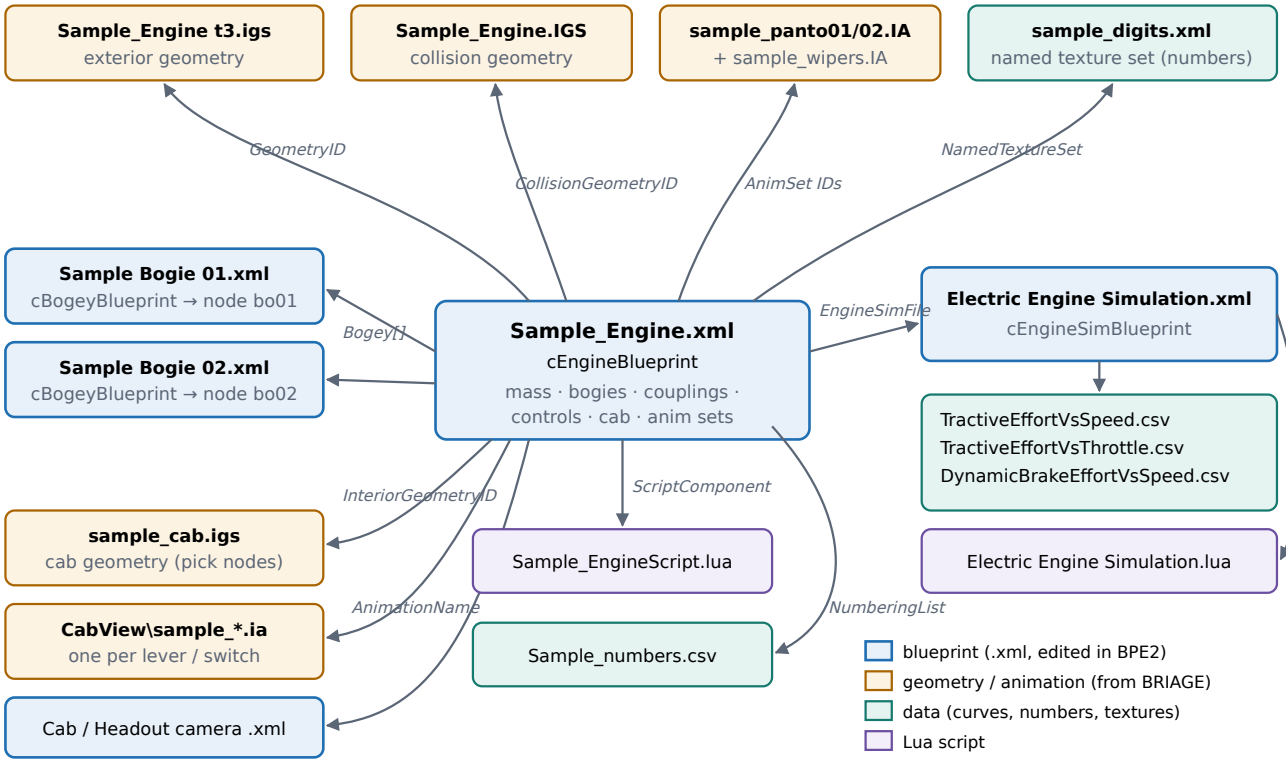


Figure 1.1 · The sample electric locomotive as the game sees it. Nothing is embedded: every arrow is a path string inside the parent file. Break one path or misspell one node name and that feature silently disappears (or the export fails). The same `.igs` holds the body, bogies and wheels; the bogie blueprints refer to their parts by **node name** (`bo01` , `bo01wh01` ...), not by file.

> **Key idea**
>
> Your Blender scene produces only the orange boxes. Everything blue, green and purple is authored in BPE2, a text editor or a Lua editor. The game joins them by **names**: file paths, node names, animation IDs and control names. Most “it doesn't work” problems are a broken name somewhere in this graph.

2 · The components inside a vehicle blueprint

A vehicle blueprint is an **entity** made of **components**. Each component is a top-level `<Attribute>` in the XML. This is the list from the sample electric, and what the game does with each one. verified

Component (XML attribute)	What it holds	What the game does with it
`RenderComponent` cAnimObjectRenderBlueprint	Exterior `GeometryID` (.igs), `CollisionGeometryID`, primary/secondary named texture sets (number digits), shadow type, LOD range and the `AnimSet` list (ID → .ia file, “animate in editor”).	Draws the vehicle; plays exterior animations by ID.
`RailVehicleComponent` cEngineComponentBlueprint (engine)	Mass, derailment ease, body pivots, collision box, drag, rolling friction, wheel-rail friction (dry/wet/snow), sand multiplier, `Bogey[]` list, coupling blueprints and pivot matrices, driver position, `EngineType`, `MaxPermissibleSpeed`, numbering CSV.	Puts the vehicle on the track and moves it as a rigid body.
`EngineSimulationContainer`	Path to the **engine simulation blueprint** (traction, brakes, air system, dynamic brake, curves).	Turns control values into tractive and brake force. Engines only.
`ControlContainerComponent`	`InteriorInterface` (cab .igs, cab camera, single/double cab) and the `ControlValues` list, each with its interface elements (levers, buttons, needles, exterior animations, visibility objects).	Creates the named controls and wires cab clicks and keys to them.
`ScriptComponent`	Path to a Lua script (compiled to `.out` on export).	Runs your custom logic (Part 5).
`ContainerComponent` cEntityContainerBlueprint	Child entities with a name and matrix: lights, particle emitters (smoke, sparks), sound sources.	Spawns attached sub-objects that scripts can address by child name.
`PosOriComponent`	Position/orientation holder (empty in the samples).	Engine-managed; nothing to author.

The **engine simulation blueprint** is its own file with one **subsystem**, whose class decides the powertrain:

Subsystem class	Powertrain	Sample
`cElectricSubSystemBlueprint`	Electric (pantograph or 3rd rail). Current limits, thyristor/notched control, dynamic brake.	Sample Electric: 6600 / 300, 145 mph
`cDieselElectricSubSystemBlueprint`	Diesel engine → generator → traction motors; field divert stages, fuel use.	(EMD F7A in the handoff doc)
`cSteamSubSystemBlueprint`	Full steam model: firebox, boiler, safety valves, cylinders, injectors, blower, ejectors, regulator.	SDJR 7F: 200 psi, 2 × 21″×28″

3 · How the game places a vehicle on the track

TSC treats the body as a **rigid object** carried by its bogie pivots. Bogies follow the track spline and turn about their pivot points. Wheels are just nodes that spin at a rate derived from speed and

wheel radius. Couplers are points at each end that lock to the neighbouring vehicle's receiving point. The drawing below uses the sample electric's real numbers.

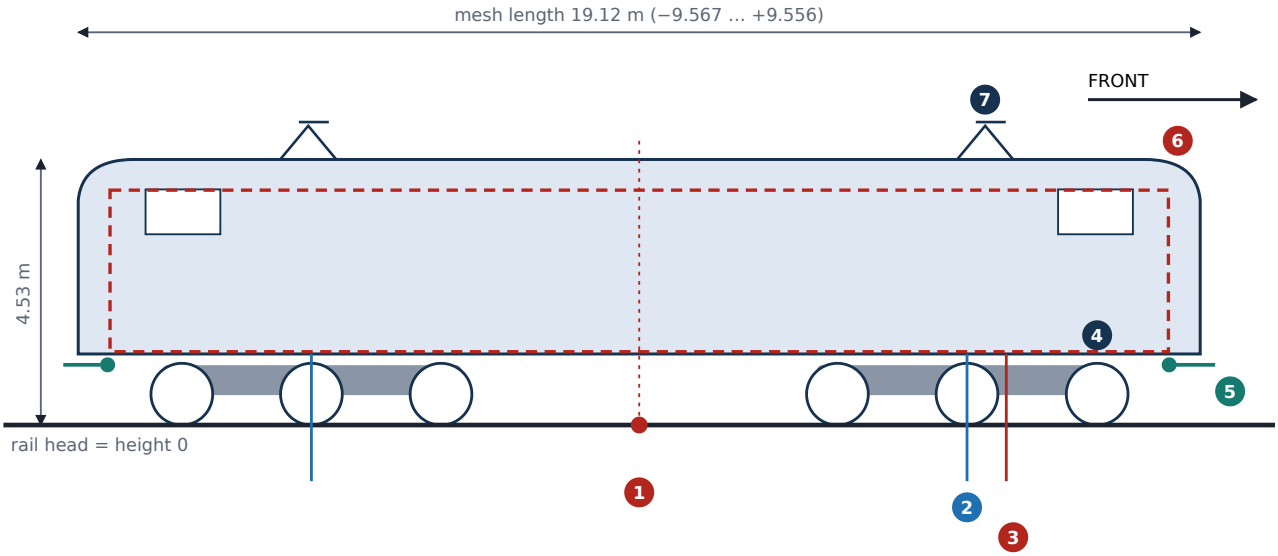


Figure 1.2 · Side view of the sample electric (front to the right), to scale. 1 Vehicle origin: centre of the vehicle, on the rail head. Everything is measured from here. **2** Front bogie as modelled (`bo01` at +5.583 m). **3** Front bogie pivot as the blueprint defines it (`BogeyPivotX` = 6.25 m). The game uses this one. **4** Axles at ±2.2 m from the bogie centre (bogie blueprint `HorizontalOffset`); radius 0.55 m in the blueprint, 0.525 m in the mesh. **5** Coupling pivot 1.025 m high at +9.025 m; receiving point at +9.14 m. **6** Collision box (dashed): 18 m long, 1.25–4.0 m high. Shorter and lower than the mesh. **7** Pantographs (animated nodes `panto01_part01..03`). verified

Field	Where it lives	Meaning
`BogeyPivotX` / `BogeyPivotY`	Vehicle bp → `Bogey[]`	Where each bogie pivots (along track / height). **Has the final say** over the mesh position.
`Geometry` , `Axle[].NodeId`	Bogie bp	Node names of the bogie frame and each wheelset inside the vehicle's .igs.
`WheelRadius` , `Axle[].Radius` , `HorizontalOffset`	Bogie bp	Radius drives wheel spin rate and sound; offset is along the track from the bogie pivot.
`FrontPivotX` / `BackPivotX`	Vehicle bp	Body pivot points (±9.515 m on the electric, +6.6/−4.185 on the steam). check How the game weighs these against bogie pivots is undocumented; keep them near the vehicle ends as the samples do.
`Front/RearCouplingPivot` , `…ReceivingPoint`	Vehicle bp (4×4 matrices)	Coupler position and direction. The rear matrix is rotated 180° (diagonal −1,1,−1). verified
`Collision*`	Vehicle bp	Box for collisions and derailment; `CollisionCentreX` is along track, `…Y` is height of the box centre.
`DriverPosition`	Vehicle bp (matrix)	Where the driver figure/camera anchor sits.

4 · Three coordinate systems and how they map

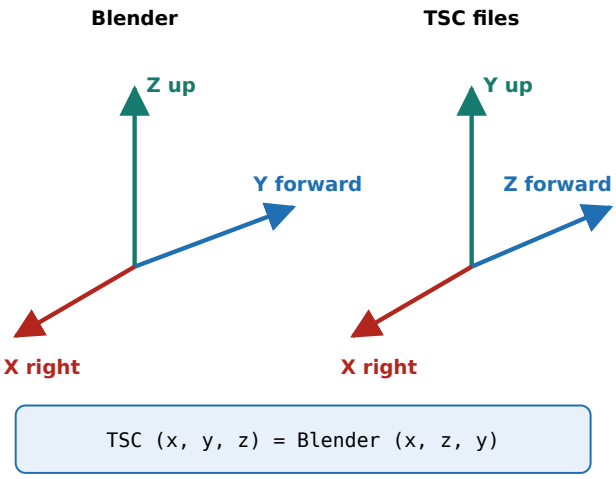


Figure 1.3 · Axis mapping. BRIAGE performs this swap on export; the sample's 33 transforms match Blender exactly after it. verified

The third system: blueprint 2D fields

Many vehicle blueprint fields describe a side view and use only two axes:

Blueprint	TSC file	Blender
X (along track)	Z	Y
Y (height)	Y	Z

So `BogeyPivotX = 6.25` means "6.25 m forward of the origin", which is Blender **Y** = 6.25.

Matrices (`FrontCouplingPivot` , `DriverPosition`) are full 4×4 transforms in **TSC file axes**, row-major, with the translation in elements 13–15. Example from the sample: `… 0, 1.025, 9.025, 1` = 1.025 m up, 9.025 m forward. verified

5 · Control values: the bus everything talks through

A **control value** is a named floating-point number on a vehicle, declared in the vehicle blueprint with a default, minimum and maximum. It is the single most important concept for custom behaviour: **every part of a vehicle communicates only by reading and writing control values.**

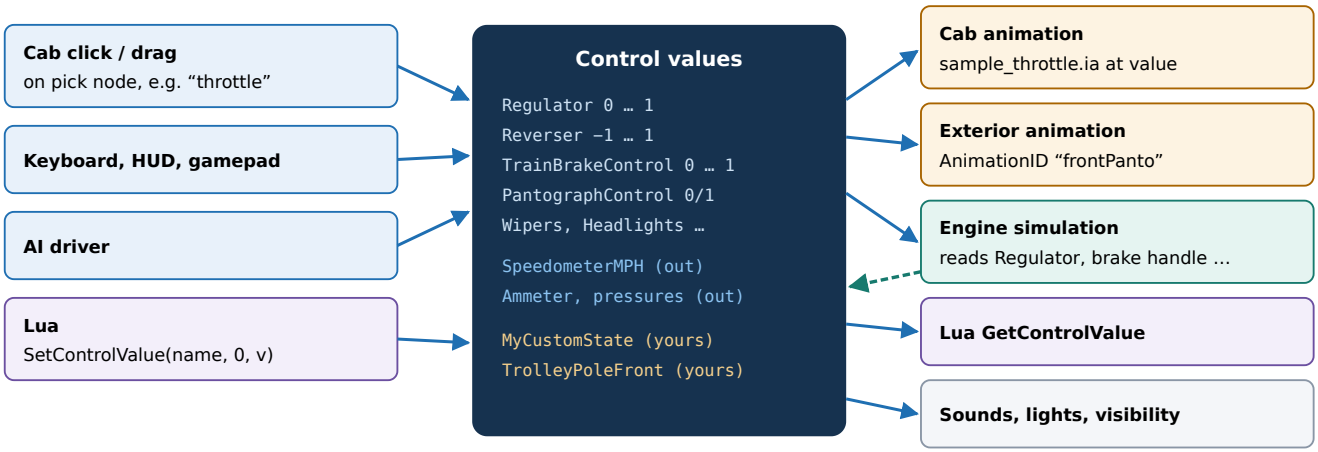


Figure 1.4 · The control-value bus. Inputs on the left write; consumers on the right read. Nothing talks to anything else directly. A cab lever does not “know” about traction: it only changes `Regulator`, and the simulation reads `Regulator`. That is why you can add features by **adding your own control values** (yellow) and driving animations or logic from them.

Rules of the bus

- **Names are the contract.** The simulation finds its inputs by name. The brake is explicit: the simulation's `CabControlName` must equal a `ControlName` in the vehicle blueprint (`TrainBrakeControl` in both samples) verified. Throttle and reverser use the conventional names `Regulator` and `Reverser`; keep them check.
- **Range maps to animation.** A cab lever's `.ia` is played *scrubbed to the value*: minimum at the first frame, maximum at the last. Model and key every lever from its minimum position to its maximum.
- **Defaults are spawn state.** `DefaultValue` is the value at spawn. The sample electric's `TrainBrakeControl` default of 0.7 means it spawns braked; the steam example's 1.0 means fully applied. verified
- **Custom controls are free.** A control value does not need a lever. Declare `TrolleyPoleFront` with range 0–1, point an exterior animation at it, and let Lua write it. This is the backbone of every scripted feature in Part 5.
- **Difficulty filters elements.** Interface elements carry a `DifficultyType` (StopGo / Intermediate / Expert), which is how the sample has both `SimpleThrottle` and `Regulator` on the same physical lever.

6 · The traction and braking chain

You do not program physics directly; you give the subsystem limits and lookup curves. This is the working model to design against. The “minimum of three limits” rule comes from the project's handoff document and is consistent with both samples. check Verify final behaviour in game with Lua (`Call("GetTractiveEffort")`) rather than trusting the model.

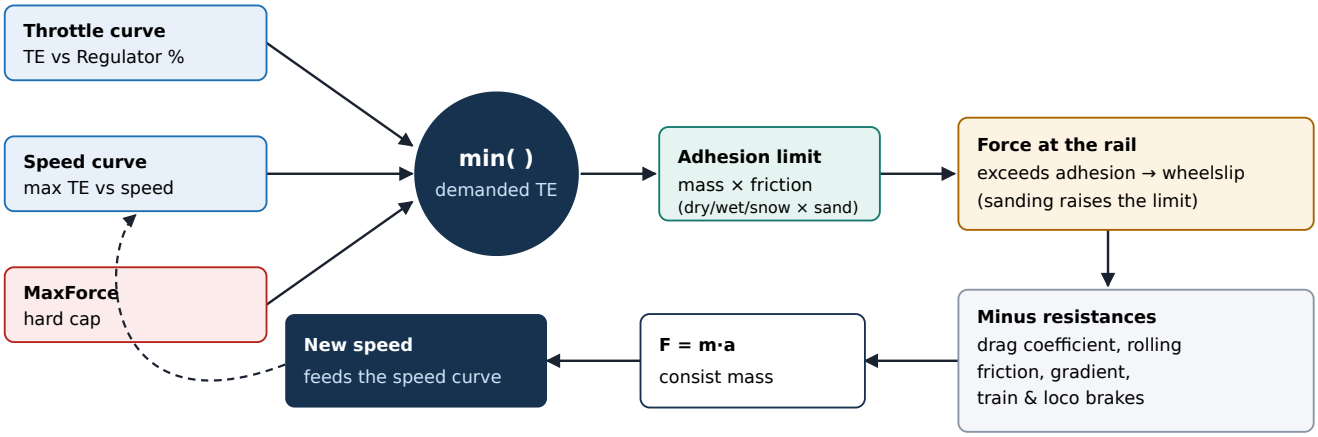


Figure 1.5 · Traction chain for electric and diesel-electric subsystems. Steam replaces the two curves with `TractiveEffortVsCutoff` (a 0–1 multiplier over cutoff, see the 7F sample) and `TractiveEffortLinearVsSpeed`, with boiler pressure and regulator opening governing how much steam is available.

Design rules that follow

- MaxForce must be at least the peak of the throttle and speed curves, or the top of the curves is unreachable. The sample electric does this exactly (throttle curve 0→300, MaxForce 300). verified
- The throttle curve must start at 0,0 or the vehicle creeps at zero throttle.
- End the speed curve at or beyond MaxSpeed.
- Mass and friction decide whether that force can be used at all.

Unit labels are not conversions

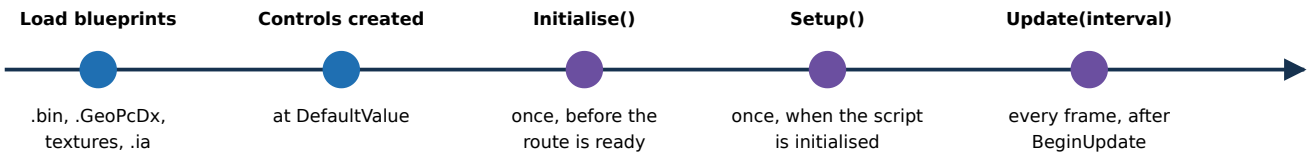
BPE2 stores a `mUnits` label next to what you typed, but the samples show the stored value equals the typed value (the 7F's MaxForce is typed as 35.296 “Lb/f”, stored as 35.296; the real engine's tractive effort is 35,296 lbf). Treat labels as hints, compare with a known-good vehicle, and measure in game. verified

7 · Engine vs wagon vs tender

Engine (drivable)	Wagon / coach (hauled)	Tender (steam only)
✔ Render (geometry, animations)	✔ Render (geometry, animations)	✔ Render, rail vehicle
✔ Rail vehicle (mass, bogies, couplers)	✔ Rail vehicle (mass, bogies, couplers)	✔ Water and coal capacity
✔ Engine simulation blueprint	✔ Train brake equipment	✔ Feeds the engine's "TenderRequired"
✔ Controls + cab interior	✘ No engine simulation	The 7F sim sets TenderRequired = eTrue
✔ Script (optional but usual)	✘ No cab (normally)	New ▸ Tender blueprint
✔ Children: lights, emitters	○ Script optional (doors, loads)	
New ▸ Engine blueprint cEngineBlueprint	New ▸ Wagon blueprint (verify class name in BPE2)	

Figure 1.6 · What each vehicle type carries. A wagon is an engine without the engine: same rendering, bogies, couplers and brakes, no powertrain, no controls to drive. Everything in Part 2 (modelling) and Part 3 (exterior animations) applies to both. check Wagon and tender field names: confirm against a stock wagon/tender blueprint in your BPE2.

8 · What happens when a scenario loads



Event callbacks fire in between: OnControlValueChange, OnConsistMessage, OnCameraEnter/Leave, OnCustomSignalMessage

Figure 1.7 · Load order. Callback names and meanings from the TS2015 scripting reference; details in Part 5.

9 · The glue: names

Summarising the whole part in one table. Every row is a link that must match **exactly** (case included):

This name…	…must match	If it doesn't
Bogie bp `Geometry` (`bo01`)	Node in exterior .igs	Bogie drawn but never turns
Bogie bp `Axle[].NodeId`	Wheel node in exterior .igs	Wheels do not spin
Control element `PickTransformName`	Node in cab .igs	Lever cannot be clicked
Control element `AnimationName`	An .ia file path	Lever works but does not move
Exterior element `AnimationID`	An ID in the RenderComponent `AnimSet`	Pantograph/wiper never moves

This name…	…must match	If it doesn't
Visibility element `TransformName`	Node in cab or exterior .igs	Lamp never shows (sample: `hs_aus_lit` is missing) **verified**
Sim `CabControlName`	A `ControlName`	Brake handle does nothing
Lua `Call("X:…")` child name	A child in `ContainerComponent`	Call silently fails
Lua control strings	A `ControlName`	`GetControlValue` returns nothing useful

> **Automate this table**
>
> `tools/tsc_crosscheck.py` checks rows 1–7 against your exported geometry in one command (Part 7). On the sample electric it reports the two missing cab nodes, the bogie offset, the wheel height and the short collision box.